Meeting Log

Project: E-Commerce & Order Management System (Olist) · **Group:** TarushTanishkTanishq
Meetings: online (about 15 min stand-ups; longer sprint reviews) · **Recorded by:** Tarush Banke (Scrum master)

Summary

#	Date	Type	Present	Key decision
M1	18 Sep	Kick-off	All	Olist dataset; PostgreSQL + Flask + vanilla JS; Scrum, 4 sprints
M2	19 Sep	Sprint 0 review	All	ER model with 16 tables accepted; composite key for `review`
M3	21 Sep	Sprint 1 mid-point	All	Business rules in triggers/procedures; 5 roles
M4	23 Sep	Sprint 1 review + retro	All	ML task changed to late-delivery prediction
M5	25 Sep	Sprint 2 mid-point	All	API contract (`data` + `queries` log); model accepted
M6	27 Sep	Sprint 2 review + retro	All	All pages working end to end
M7	29 Sep	Sprint 3 check-in	All	Fixed D-01/D-02; plan report and video

Meeting minutes

M1: Kick-off (18 Sep) · Present: all - Chose the Olist dataset for the e-commerce topic (real data, 9 related tables). - Agreed on PostgreSQL + Flask + vanilla JS; Scrum with 4 sprints. - Actions: Tarush sets up the repo and schema draft; Tanishq profiles the CSVs; Tanishk checks PostgreSQL features for security (RLS).

M2: Sprint 0 review (19 Sep) · Present: all - ER model accepted: 16 tables, `review` needs a composite key because `review_id` repeats. - Profiling: 8.1% of orders are late, late orders average 2.57★ vs 4.29★. - Actions: Tarush writes the staging load and transform; Tanishk starts the triggers.

M3: Sprint 1 mid-point (21 Sep) · Present: all - Decided to put business rules in triggers and procedures, not in Flask. - Agreed on 5 application roles mapped to PostgreSQL group roles. - Actions: Tanishk writes the security script; Tarush writes indexes and views.

M4: Sprint 1 review + retro (23 Sep) · Present: all - Database builds from scratch in about 1 minute. ✔ - Retro: loading took longer than planned because of dirty geolocation data. Next time we profile before writing the DDL. - ML task changed to late-delivery prediction (recommendation not viable, R2). - Actions: Tanishq trains the model; Tanishk builds the API; Tarush writes the feature view.

M5: Sprint 2 mid-point (25 Sep) · Present: all - API contract agreed: every response returns `data` plus a `queries` log for the SQL panel. - Model results: ROC-AUC 0.72 on held-out months; accepted with an honest caveat about concept drift.

M6: Sprint 2 review + retro (27 Sep) · Present: all - All pages working end to end with the SQL panel. - Retro: integration was smooth because of the agreed API contract. - Actions: Tanishk adds the transaction/concurrency demos; Tanishq writes the test plan and screenshots; Tarush writes the docs.

M7: Sprint 3 check-in (29 Sep) · Present: all - Fixed: the home page `/` returned 404 and the app gave an unclear error when PostgreSQL was not running. - Actions: finish the report, record the demo video, and each member prepares to explain their part.
